**Assignment (Module IV): Core Java**

**Task 1: Library Book Management – Using ArrayList**
**Scenario:**
**A library maintains a list of book titles. The system should allow adding new books, removing books, and displaying the complete book list.**
**Program Structure:**
**• Class: Library**
**o Use ArrayList<String> to store book titles.**
**o Methods: addBook(), removeBook(), displayBooks().**
**• Menu-driven program.**

**Solution :**

```
import java.util.ArrayList;
import java.util.Scanner;

public class Library {
    private final ArrayList<String> books = new ArrayList<>();

    void addBook(String title) {
        books.add(title);
        System.out.println("Book added successfully.");
    }

    void removeBook(String title) {
        if (books.remove(title))
            System.out.println("Book removed successfully.");
        else
            System.out.println("Book not found.");
    }

    void displayBooks() {
        System.out.println("Current Books:");
        if (books.isEmpty()) {
            System.out.println("No books available.");
            return;
        }
        for (String book : books)
```

```
            System.out.println(book);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library library = new Library();

        while (true) {
            System.out.println("\n1. Add Book");
            System.out.println("2. Remove Book");
            System.out.println("3. Display All Books");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int choice = Integer.parseInt(sc.nextLine());

            switch (choice) {
                case 1:
                    System.out.print("Enter book title: ");
                    library.addBook(sc.nextLine());
                    break;
                case 2:
                    System.out.print("Enter book title to remove: ");
                    library.removeBook(sc.nextLine());
                    break;
                case 3:
                    library.displayBooks();
                    break;
                case 4:
                    sc.close();
                    return;
                default:
                    System.out.println("Invalid choice.");
            }
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT751_
ExceptionMessages' '-cp' 'C:\Users\le
s\CSIT751_Module4_Java_Programs_Only_

1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 1
Enter book title: Data Structures
Book added successfully.
```

```
1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice: 3
Current Books:
Data Structures

1. Add Book
2. Remove Book
3. Display All Books
4. Exit
Enter choice:
```

**Task 2: Student Attendance System – Using HashSet**
**Scenario:**
**In a classroom, attendance must not contain duplicates. Using a Set ensures each student**
**is marked only once.**
**Program Structure:**
**• Class: Attendance**
**o Use HashSet<String> to store student names.**
**o Methods: markAttendance(), displayAttendance().**
**• Prevent duplicate entries.**

**Solution :**

```
import java.util.HashSet;
import java.util.Scanner;

public class Attendance {
    private final HashSet<String> students = new HashSet<>();

    void markAttendance(String name) {
        if (students.add(name))
            System.out.println("Attendance marked for " + name);
        else
            System.out.println(name + " is already marked present");
    }

    void displayAttendance() {
        System.out.println("Attendance List:");
        for (String student : students)
            System.out.println(student);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Attendance attendance = new Attendance();

        System.out.print("Enter name to mark attendance: ");
        attendance.markAttendance(sc.nextLine());

        System.out.print("Enter name to mark attendance: ");
        attendance.markAttendance(sc.nextLine());

        attendance.displayAttendance();
        sc.close();
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT751_Modu
ShowCodeDetailsInExceptionMessages' '-cp'
1cd5f2da091ecb33\redhat.java\jdt_ws\CSIT7
Enter name to mark attendance: Rohan
Attendance marked for Rohan
Enter name to mark attendance: Rohan
Rohan is already marked present
Attendance List:
Rohan
PS C:\Users\lenovo\Downloads\CSIT751_Modu
```

**Task 3: Bank Account Directory – Using HashMap**
**Scenario:**
**A bank maintains a directory mapping account numbers to customer names. It should allow adding accounts, retrieving customer name by account number, and displaying all entries.**
**Program Structure:**
**• Class: BankDirectory**
**o Use HashMap<Integer, String>.**
**o Methods: addAccount(), getCustomer(), displayAll().**

**Solution :**

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class BankDirectory {
    private final HashMap<Integer, String> accounts = new HashMap<>();

    void addAccount(int accountNo, String customerName) {
        accounts.put(accountNo, customerName);
        System.out.println("Account added successfully.");
    }

    void getCustomer(int accountNo) {
        String customer = accounts.get(accountNo);
        if (customer != null)
            System.out.println("Customer Name: " + customer);
        else
            System.out.println("Account not found.");
    }

    void displayAll() {
        if (accounts.isEmpty()) {
            System.out.println("No accounts available.");
            return;
        }
        for (Map.Entry<Integer, String> entry : accounts.entrySet())
            System.out.println("Account No: " + entry.getKey() + " -> " +
entry.getValue());
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankDirectory directory = new BankDirectory();
```

```
        while (true) {
            System.out.println("\n1. Add Account");
            System.out.println("2. Get Customer Name");
            System.out.println("3. Display All Accounts");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int choice = Integer.parseInt(sc.nextLine());

            switch (choice) {
                case 1:
                    System.out.print("Enter Account No: ");
                    int accountNo = Integer.parseInt(sc.nextLine());
                    System.out.print("Enter Customer Name: ");
                    String name = sc.nextLine();
                    directory.addAccount(accountNo, name);
                    break;
                case 2:
                    System.out.print("Enter Account No: ");

directory.getCustomer(Integer.parseInt(sc.nextLine()));
                    break;
                case 3:
                    directory.displayAll();
                    break;
                case 4:
                    sc.close();
                    return;
                default:
                    System.out.println("Invalid choice.");
            }
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT
ShowCodeDetailsInExceptionMessage
1cd5f2da091ecb33\redhat.java\jdt_

1. Add Account
2. Get Customer Name
3. Display All Accounts
4. Exit
Enter choice: 1
Enter Account No: 101
Enter Customer Name: Ankit
Account added successfully.
```

```
1. Add Account
2. Get Customer Name
3. Display All Accounts
4. Exit
Enter choice: 3
Account No: 101 -> Ankit

1. Add Account
2. Get Customer Name
3. Display All Accounts
4. Exit
Enter choice:
```

**Task 4: Product Inventory Management – Using HashMap and Iterator**
**Scenario:**
**A shop maintains an inventory where each product ID maps to its stock count. The program should add products, update stock, and display inventory using an iterator.**
**Program Structure:**
**• Class: Inventory**
**o Use HashMap<Integer, Integer> for product ID → stock.**
**o Methods: addProduct(), updateStock(), displayInventory().**

**Solution :**

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Scanner;

public class Inventory {
    private final HashMap<Integer, Integer> products = new HashMap<>();

    void addProduct(int productId, int stock) {
        products.put(productId, stock);
        System.out.println("Product " + productId + " added with stock " +
stock);
    }

    void updateStock(int productId, int stock) {
        if (products.containsKey(productId)) {
            products.put(productId, stock);
            System.out.println("Stock updated successfully.");
        } else {
            System.out.println("Product not found.");
        }
    }

    void displayInventory() {
        Iterator<Map.Entry<Integer, Integer>> iterator =
products.entrySet().iterator();
        if (!iterator.hasNext()) {
            System.out.println("Inventory is empty.");
            return;
        }
        while (iterator.hasNext()) {
            Map.Entry<Integer, Integer> e = iterator.next();
            System.out.println("Product ID: " + e.getKey() + ", Stock: " +
e.getValue());
        }
```

```
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Inventory inventory = new Inventory();

        while (true) {
            System.out.println("\n1. Add Product");
            System.out.println("2. Update Stock");
            System.out.println("3. Display Inventory");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            int choice = Integer.parseInt(sc.nextLine());

            switch (choice) {
                case 1:
                    System.out.print("Enter Product ID: ");
                    int id = Integer.parseInt(sc.nextLine());
                    System.out.print("Enter Stock: ");
                    int stock = Integer.parseInt(sc.nextLine());
                    inventory.addProduct(id, stock);
                    break;
                case 2:
                    System.out.print("Enter Product ID: ");
                    int updateId = Integer.parseInt(sc.nextLine());
                    System.out.print("Enter New Stock: ");
                    int newStock = Integer.parseInt(sc.nextLine());
                    inventory.updateStock(updateId, newStock);
                    break;
                case 3:
                    inventory.displayInventory();
                    break;
                case 4:
                    sc.close();
                    return;
                default:
                    System.out.println("Invalid choice.");
            }
        }
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT7
ShowCodeDetailsInExceptionMessages
1cd5f2da091ecb33\redhat.java\jdt_w

1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 1
Enter Product ID: 101
Enter Stock: 50
Product 101 added with stock 50
```

```
1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice: 3
Product ID: 101, Stock: 50

1. Add Product
2. Update Stock
3. Display Inventory
4. Exit
Enter choice:
```

**Task 5: Online Course Enrollment – Using LinkedHashSet**
**Scenario:**
**Students enroll in online courses. The enrollment list must be stored in the order of registration but without duplicates. LinkedHashSet is ideal here.**
**Program Structure:**
**• Class: CourseEnrollment**
**o Use LinkedHashSet<String>.**
**o Methods: enrollStudent(), displayEnrolledStudents().**

**Solution :**

```
import java.util.LinkedHashSet;
import java.util.Scanner;

public class CourseEnrollment {
    private final LinkedHashSet<String> students = new LinkedHashSet<>();

    void enrollStudent(String name) {
        if (students.add(name))
            System.out.println("Enrolled: " + name);
        else
            System.out.println(name + " is already enrolled");
    }

    void displayEnrolledStudents() {
        System.out.println("Enrolled Students: " + students);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CourseEnrollment enrollment = new CourseEnrollment();

        for (int i = 0; i < 3; i++) {
            System.out.print("Enroll: ");
            enrollment.enrollStudent(sc.nextLine());
        }

        enrollment.displayEnrolledStudents();
```

```
        sc.close();
    }
}
```

```
PS C:\Users\lenovo\Downloads\CSIT751_Module4_Ja
ShowCodeDetailsInExceptionMessages' '-cp' 'C:\U
1cd5f2da091ecb33\redhat.java\jdt_ws\CSIT751_Mod
Enroll: Aditi
Enrolled: Aditi
Enroll: Rohan
Enrolled: Rohan
Enroll: Aditi
Aditi is already enrolled
Enrolled Students: [Aditi, Rohan]
```

**Task 6: Student Registration Form — GUI with AWT Components**

***Scenario:***

**Create a student registration form using AWT components. The form should accept basic student details and display them after submission.**

***Task:***

**• Use Frame as container.**
**• Add labels and text fields for Name, Roll No, and Course.**
**• Add a Submit button.**
**• On click, display the entered data in a dialog box or on the console.**
**• Use FlowLayout.**

**Solution :**

```
import java.awt.*;
import java.awt.event.*;

public class StudentRegistrationForm extends Frame {

    TextField name, father, age;
    CheckboxGroup gender = new CheckboxGroup();
    Checkbox male, female, drawing, singing, music, others;
    Choice course = new Choice();
    TextArea address;
    Button save, clear;

    StudentRegistrationForm() {
        setTitle("User Registration Form");
        setSize(830, 500);
        setLayout(null);
        setBackground(new Color(54, 61, 75));

        Label title = L("Registration Form", 280, 45, 270);
        title.setAlignment(Label.CENTER);
        title.setForeground(Color.YELLOW);
        title.setFont(new Font("Arial", Font.BOLD, 20));

        L("Name", 205, 95, 100);
        name = T(330, 95);

        L("Father Name", 205, 140, 100);
        father = T(330, 140);

        L("Age", 205, 185, 100);
        age = T(330, 185);

        L("Gender", 205, 230, 100);
```

```
        male = G("Male", 330, 230);
        female = G("Female", 420, 230);

        L("Course", 205, 275, 100);
        for (String s : new String[]{"Select
Course","Java","Python","C","C++","HTML","JavaScript"})
            course.add(s);
        course.setBounds(330, 275, 340, 30);
        add(course);

        L("Hobbies", 205, 320, 100);
        drawing = C("Drawing", 330, 320, 90);
        singing = C("Singing", 420, 320, 80);
        music = C("Music", 500, 320, 80);
        others = C("Others", 580, 320, 90);

        L("Address", 205, 360, 100);
        address = new TextArea();
        address.setBounds(330, 360, 340, 80);
        add(address);

        save = B("Save Details", 330, Color.BLUE);
        clear = B("Clear All", 470, Color.RED);

        save.addActionListener(e -> saveData());
        clear.addActionListener(e -> clearData());

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });

        setLocationRelativeTo(null);
        setVisible(true);
    }

    Label L(String s, int x, int y, int w) {
        Label l = new Label(s);
        l.setBounds(x, y, w, 25);
        l.setForeground(Color.WHITE);
        add(l);
        return l;
    }

    TextField T(int x, int y) {
        TextField t = new TextField();
```

```
        t.setBounds(x, y, 340, 28);
        add(t);
        return t;
    }

    Checkbox G(String s, int x, int y) {
        Checkbox c = new Checkbox(s, gender, false);
        c.setBounds(x, y, 90, 25);
        c.setForeground(Color.WHITE);
        add(c);
        return c;
    }

    Checkbox C(String s, int x, int y, int w) {
        Checkbox c = new Checkbox(s);
        c.setBounds(x, y, w, 25);
        c.setForeground(Color.WHITE);
        add(c);
        return c;
    }

    Button B(String s, int x, Color color) {
        Button b = new Button(s);
        b.setBounds(x, 450, 130, 30);
        b.setBackground(color);
        b.setForeground(Color.WHITE);
        add(b);
        return b;
    }

    void saveData() {
        Checkbox g = gender.getSelectedCheckbox();
        String gen = g == null ? "Not Selected" : g.getLabel();

        String h = "";
        if (drawing.getState()) h += "Drawing ";
        if (singing.getState()) h += "Singing ";
        if (music.getState()) h += "Music ";
        if (others.getState()) h += "Others ";
        if (h.isEmpty()) h = "None";

        showResult(name.getText(), father.getText(), age.getText(),
                gen, course.getSelectedItem(), h, address.getText());
    }

    void clearData() {
        name.setText("");
```

```
        father.setText("");
        age.setText("");
        address.setText("");
        gender.setSelectedCheckbox(null);
        course.select(0);
        drawing.setState(false);
        singing.setState(false);
        music.setState(false);
        others.setState(false);
    }

    void showResult(String n, String f, String a, String g,
                    String c, String h, String ad) {

        Dialog d = new Dialog(this, "Registration Successful", true);
        d.setSize(500, 430);
        d.setLayout(null);

        Label head = new Label("Student Registered Successfully!",
Label.CENTER);
        head.setBounds(50, 50, 400, 35);
        head.setFont(new Font("Arial", Font.BOLD, 17));
        head.setForeground(new Color(0, 128, 0));
        d.add(head);

        String[] data = {
            "Name          : " + n,
            "Father Name   : " + f,
            "Age           : " + a,
            "Gender        : " + g,
            "Course        : " + c,
            "Hobbies       : " + h
        };

        int y = 105;
        for (String s : data) {
            Label l = new Label(s);
            l.setBounds(70, y, 360, 25);
            d.add(l);
            y += 35;
        }

        Label al = new Label("Address       :");
        al.setBounds(70, 315, 100, 25);
        d.add(al);

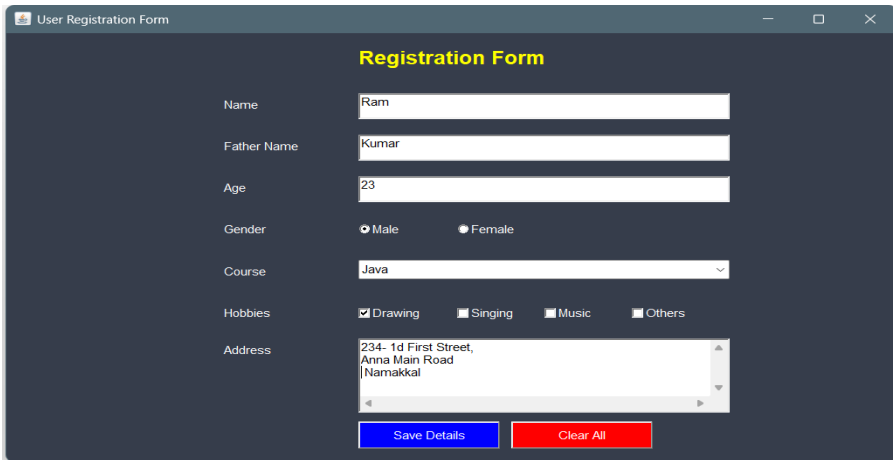
        TextArea ta = new TextArea(ad);
```

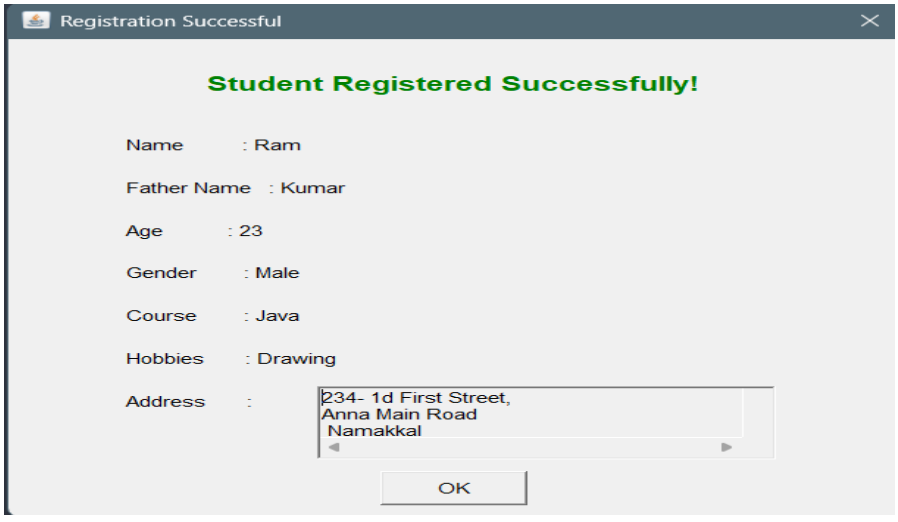
```
        ta.setBounds(175, 315, 250, 60);
        ta.setEditable(false);
        d.add(ta);

        Button ok = new Button("OK");
        ok.setBounds(210, 385, 80, 28);
        ok.addActionListener(e -> d.dispose());
        d.add(ok);

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
        new StudentRegistrationForm();
    }
}
```



**Task 7: Calculator Form — Event-Driven Programming**

***Scenario:***

**Create a simple calculator GUI to perform addition, subtraction, multiplication, and division between two numbers.**

***Task:***

**• Use JFrame, JTextField, and JButton.**

**• Add event handling for each operator button.**

**• Display the result in a text field or label.**

**Solution :**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {

    JTextField t = new JTextField();
    double n1, n2;
    String op = "";

    Calculator() {
        setTitle("Calculator");
        setSize(270, 310);
        setLayout(null);
        getContentPane().setBackground(Color.BLUE);

        t.setBounds(35, 20, 190, 30);
        add(t);

        String[] b = {
            "+","1","2","3",
            "-","4","5","6",
            "*","7","8","9",
            "/",".","0","C",
            "="
        };

        int x = 10, y = 60;

        for (int i = 0; i < b.length; i++) {
            JButton btn = new JButton(b[i]);

            if (b[i].equals("=")) {
                btn.setBounds(100, 220, 60, 35);
            } else {
```

```
                btn.setBounds(x, y, 55, 35);
                x += 60;

                if ((i + 1) % 4 == 0) {
                    x = 10;
                    y += 40;
                }
            }

            btn.addActionListener(this);
            add(btn);
        }

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {

        String s = e.getActionCommand();

        if ("0123456789.".contains(s)) {
            t.setText(t.getText() + s);
        }

        else if ("+-*/".contains(s)) {
            n1 = Double.parseDouble(t.getText());
            op = s;
            t.setText("");
        }

        else if (s.equals("=")) {
            n2 = Double.parseDouble(t.getText());
            double ans = 0;

            if (op.equals("+")) ans = n1 + n2;
            if (op.equals("-")) ans = n1 - n2;
            if (op.equals("*")) ans = n1 * n2;
            if (op.equals("/")) ans = n1 / n2;

            t.setText(n1 + op + n2 + "=" + ans);
        }

        else if (s.equals("C")) {
            t.setText("");
            n1 = n2 = 0;
```

```
            op = "";
        }
    }

    public static void main(String[] args) {
        new Calculator();
    }
}
```

